TASK 11: Final Design Recommendation for the Metro Smart-Card System

Name: Nagineni Sunil Kumar

Register Number: 192472330

Language: C++17

1. Aim

To recommend a suitable software design for the City Metro Smart-Card System by considering fare calculation, balance management, validation, accuracy, and future improvements.

2. Introduction

The City Metro Smart-Card System calculates passenger fares based on travel zones and travel time. It checks the available card balance before allowing a journey. A good design should be accurate, secure, easy to maintain, and simple to extend with new features.

3. Recommended Design

An object-oriented approach using C++ classes is recommended for the metro smart-card system.

The main class is SmartCard, which stores the card balance and provides functions to calculate fares, deduct money, and validate transactions.

Main Components

1. Fare Calculation: Calculates fares based on the selected zone.
2. Peak-Hour Calculation: Applies the 20% surcharge between 08:00–10:00 and 17:00–20:00.
3. Balance Validation: Ensures that at least ₹50 remains after the fare is deducted.

4. Input Validation: Rejects invalid zone names, times, and amounts.
5. Transaction Processing: Deducts the fare only when the transaction is valid.

4. Fare Rules

Zone	Base Fare
Inner	₹10
Middle	₹20
Outer	₹30

Peak-hour journeys receive a 20% surcharge. The minimum remaining balance must be ₹50.

5. Why Object-Oriented Programming Is Suitable

- Encapsulation: Protects the card balance from direct, unauthorized changes.
- Abstraction: Hides the internal details of fare calculation.
- Modularity: Separates fare calculation, validation, and payment.
- Maintainability: Makes the program easier to update and debug.
- Reusability: The same class can be used for many smart cards.

6. Recommended Improvements

- Store money in integer paise to avoid floating-point rounding errors.
- Validate every input before processing a transaction.
- Use suitable data types to prevent integer overflow.
- Add transaction records for checking previous journeys.
- Test peak-hour boundaries and low-balance situations.

7. Algorithm

1. Start the program.
2. Read the card balance, travel zone, and journey time.
3. Validate the input values.
4. Calculate the base fare.
5. Apply the peak-hour surcharge when applicable.
6. Check whether the remaining balance will be at least ₹50.
7. If the balance is sufficient, deduct the fare and display the result.
8. Otherwise, deny the journey and display a message.
9. Stop the program.

8. Complexity Analysis

- Time Complexity: (O(1)) for processing one journey, because the fare calculation uses a fixed number of operations.
- Space Complexity: (O(1)) for storing the details of one journey.

9. Expected Result

The recommended design provides a modular and reliable way to calculate fares, validate card balances, and process metro journeys. It also makes future changes easier.

10. Conclusion

An object-oriented C++17 design is suitable for the City Metro Smart-Card System because it improves data protection, code organization, reusability, and maintenance. Integer-based money calculations and proper input validation help make the system more accurate and reliable.